School of Engineering

Course: Diploma in Electronic and Computer Engineering
Diploma in Aerospace Systems & Management

Learning Unit : EG2E55/EG2C68 Microcontroller Programming & Interfacing

Experiment No: Lab 05

Experiment Title: ADC

Objective: To learning programming with ADC inputs.

PART 1

1. The ADC Circuit

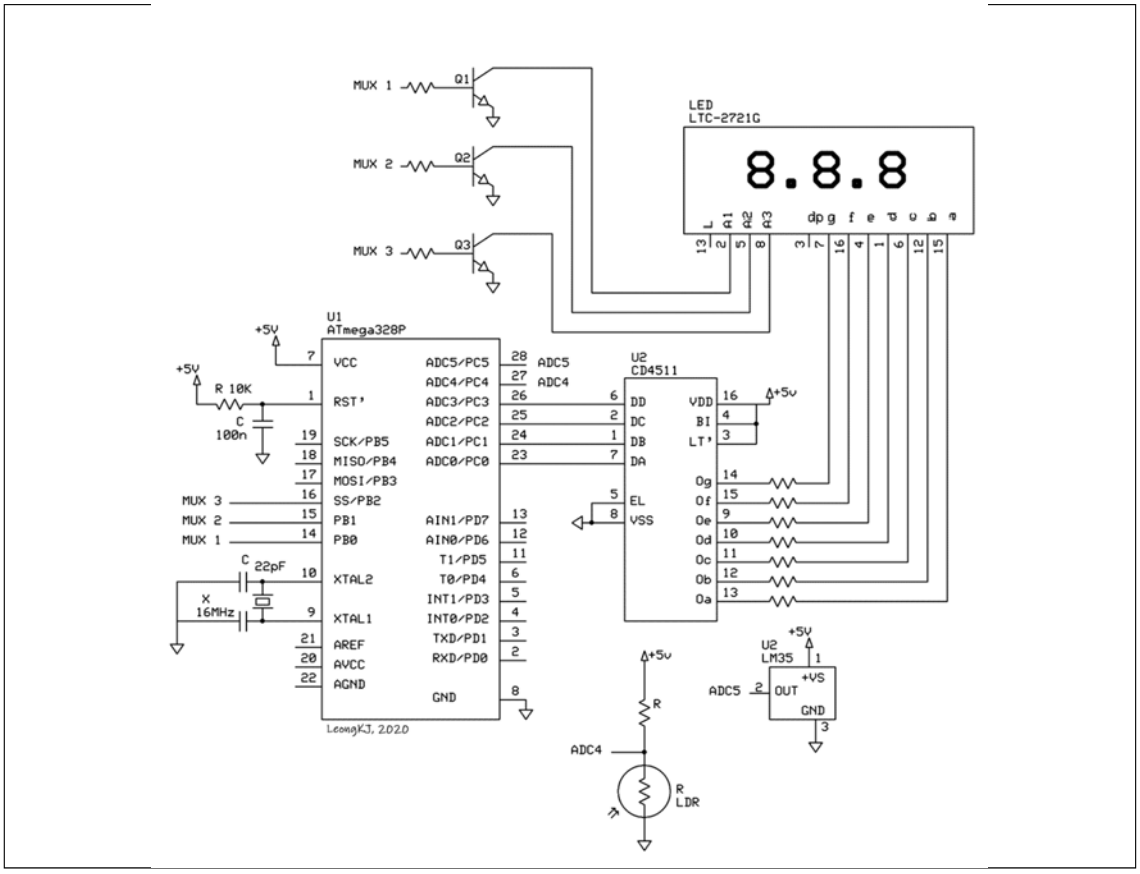


Figure 1-1: The ADC Circuit

2. Basic ADC Program

The program in Listing 2-1 reads the analog input and display its converted digital value in the 3-Digit 7-Segment LED display.

```
#include <avr/io.h>

void delay(unsigned long k){
  while(--k!=0);
}

void display(unsigned int num){
  unsigned char a,b,c,x;
  a = num/100;
  b = (num%100)/10;
  c = num%10;
  for(x=0;x<100;x++){
    PORTB = (PORTB & 0xF8)| 0x01;
    PORTC = (PORTC & 0xF0)| a;
    delay(100);
    PORTB = (PORTB & 0xF8)| 0x02;
    PORTC = (PORTC & 0xF0)| b;
    delay(100);
    PORTB = (PORTB & 0xF8)| 0x04;
    PORTC = (PORTC & 0xF0)| c;
    delay(100);
  }
}

void ADC_Init(){
   DIDR0 = 0x30;
   DDRC = DDRC & ~(0x30);
  PORTC = PORTC & ~(0x30);
  ADCSRA |= 0x87;
   ADMUX = 0xC0;
}
```

```
unsigned int ADC_Read(unsigned char c){
  ADMUX = (ADMUX & 0xF0) | c;
  ADCSRA |= 0x40; // Start ADC.
  while ((ADCSRA & 0x10)==0); // Wait.
  ADCSRA |= 0x10; // Reset.
  return ADCW;
}


int main()
{
  unsigned int adcValue;
  DDRC |= 0x0F;  // Configure output.
  DDRB |= 0x07;  // Configure output.
  ADC_Init();
  while(1){
    adcValue = ADC_Read(5);
    display(adcValue);
  }
}
```

Listing 2-1

3.1 Exercise 1

The program in Listing 2-1 display the raw digital value after the conversion and not the actual temperature as recorded by the sensor. Modify the program to display the temperature. Use Listing 3-1 as a guide.

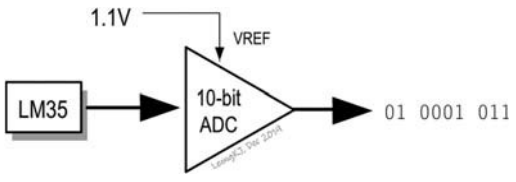


Figure 3-1

After modifying the program to display the temperature, you may realise that only the integer value of the temperature is displayed. Example, if the temperature is 31.25°C, only 31 is displayed.

Since the display is 3 digits, optimise the use of it to display up to 1 decimal place.

Note: The integer divide {/} operator will function as a float divide when handling float number.

```
float convert(unsigned int t){
  - -
  - - - -
  return(c);
}

int main(){
  float t;
  - - - -
  while(1){
    - - - -
    adcValue = ADC_Read(5);
    t = convert(adcValue);
    display(t);
  }
}
```

Listing 3-1

Part 2

3.2 Exercise 2

Develop an automatic light turn-on system (light on when darkness falls) using the Light Dependent Resistor (LDR). Turn on the LEDs connected to Port D (not shown in Figure 1-1) when the ambient light level falls. Use the pseudo code shown in Listing 3-2 as a guide.

The LDR has a large dark resistance. The dark resistance could give a ADC value of 1023 max and the light resistance would give an ADC value of less than 300. If the ADC value exceeds '999', you should cap it to '999' since your display is only 3 digits.

```
int main()
{
  DDRC |= 0x0F;  // Configure output.
  DDRB |= 0x07;
  DDRD = 0xFF; // Configure LED port.
  ADC_Init();
  while(1){
    Read ADC & display it.
    If light level is low {
      turn on LEDs.
    }
    If light level is high {
      turn off LEDs.
    }
  }
}
```

Listing 3-2

```
if (adcValue>999) adcValue=999;
```

3.3 Exercise 3

Combine your solution in exercise 1 and 2 to build a simple device that monitor the temperature and at the same time turns on the light when the ambient light is low (see Figure 3-1).

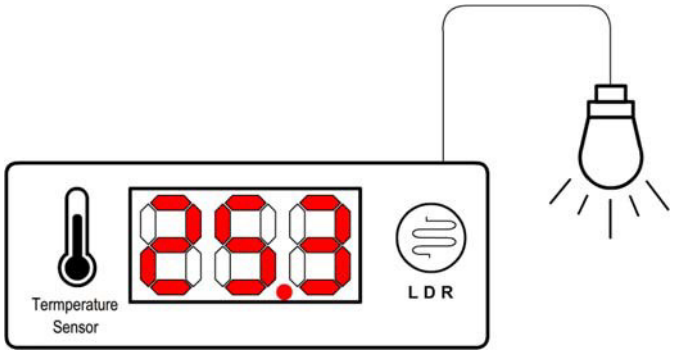


Figure 3-1

```
void ADC_Init(){
  -----
  -----
}

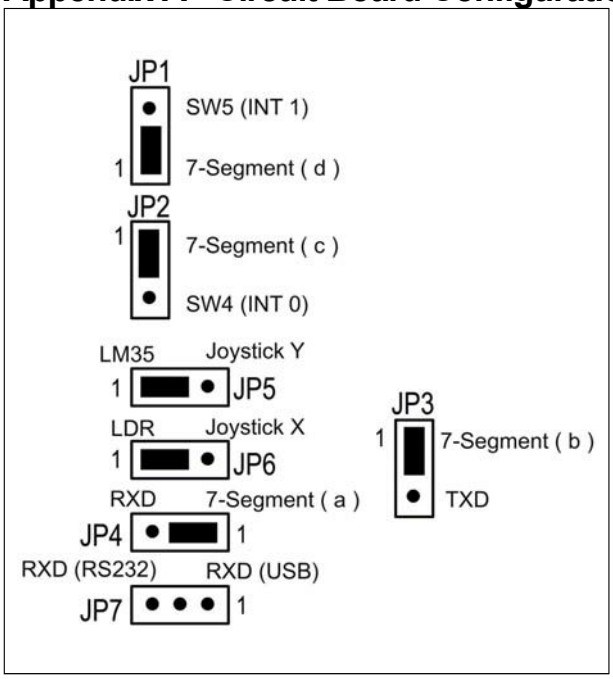
unsigned int ADC_Read(unsigned char c){
  -----
  -----
}
```

```
int main(){
 DDRC |= 0x0F;
 DDRB |= 0x07;
 DDRD = 0xFF;
 ADC_init();
 while(1){
   // Read temperature.
   // Display temperature.
   // Read light sensor.
   // Decision making instructions
   // to on/off light.

}
```

Listing 3-3

Appendix A - Circuit Board Configuration



END OF PAPER